FREE LLM ROUTER V0.7.0

Architecture, MVP Features and Routing Decisions

A presentation-ready technical and product brief for the self-hosted gateway that routes OpenAI-compatible, Codex, Claude Code and private Ollama requests through one reliability layer.

ONE-SENTENCE PITCH

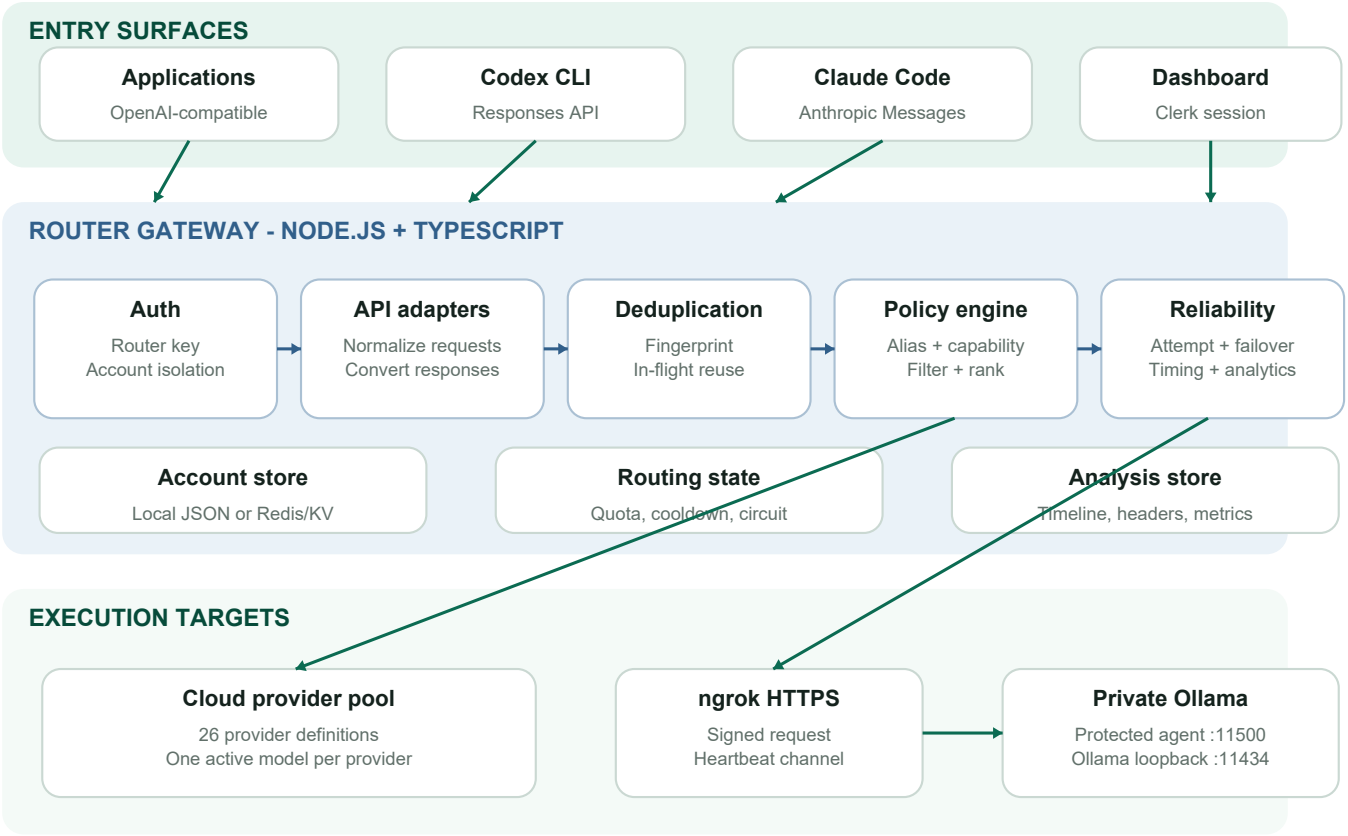
Free LLM Router gives applications one private API key for multiple cloud providers and local Ollama models, then selects a compatible, healthy and available model for each request.

Document scope

Architecture, all current MVP capabilities, every routing decision input, provider outcome behavior, practical examples, implementation boundaries and a suggested 12-slide narrative.

1. System architecture

The implementation is easiest to explain as three cooperating planes: management, routing and private execution.



Control plane

The Clerk-authenticated dashboard manages accounts, cloud provider keys, model catalogs, aliases, routing policies, quotas, reliability settings, local nodes and analytics.

Routing data plane

Clients authenticate with an flm_ router key. Requests are normalized, safely deduplicated, evaluated, ranked, attempted, converted back to the caller's protocol and recorded.

Private compute plane

The npm CLI pairs an Ollama computer. A protected loopback agent exposes only health, model discovery and inference through ngrok HTTPS. Ollama administration routes remain private.

2. Storage and security architecture

Area	Local deployment	Hosted deployment	Protection
Accounts and provider keys	**.freellm/accounts.json**	Upstash Redis/KV	AES-256-GCM when ACCOUNT_ENCRYPTION_KEY is configured; hosted storage requires it.
Routing state	**.freellm/routing-state.json**	Router-scoped Redis records	Separate provider health, round-robin cursor, quota, cooldown and circuit state.
Local nodes	**.freellm/local-nodes.json**	Owner-scoped Redis records	Single-use pairing, hashed device credentials and account ownership checks.
Analytics	**.freellm/analytics.json**	Router-scoped Redis records	Sanitized snapshots, request IDs, timelines and bounded stored content.
Deduplication	**Process memory**	Per serverless instance	Router-scoped fingerprints; streaming and unsafe requests bypass reuse.

Local-node trust boundary

- Supported path: Router -> ngrok HTTPS -> protected agent on 127.0.0.1:11500 -> Ollama on 127.0.0.1:11434.
- Pairing codes are short-lived, single-use, account-bound and stored as hashes.
- Each inference request receives a short-lived Ed25519-signed token containing node, account, request ID and model claims.
- The agent rejects replayed request IDs and enforces model allowlists, capabilities and resource limits.
- Heartbeats carry node status and model metadata, not prompts or generated responses.

3. Current MVP feature inventory

The current product surface contains 35 presentation-worthy capabilities. Each row links the feature to a practical reason it matters.

#	Feature	What exists today	Real-life value
1	**Unified AI gateway**	One base URL and router key for cloud and local models.	Switch providers without changing application integration code.
2	**Three API formats**	OpenAI Chat Completions, OpenAI Responses/Codex and Anthropic Messages/Claude Code.	A web app, Codex CLI and Claude Code share one provider pool.
3	**Self-hosted dashboard**	Overview, Providers, Playground, Analysis, Docs and Settings.	Operate the router without building an internal admin interface.
4	**Account isolation**	Each Clerk user has an isolated router configuration and private flm_ key.	Separate users cannot access each other's credentials or nodes.
5	**Bring your own keys**	Upstream provider credentials remain behind the router.	Applications receive one router key instead of every provider secret.
6	**26-provider catalog**	Cloud providers include OpenRouter, Groq, NVIDIA, Cerebras, Mistral, Gemini, OpenAI, Anthropic and others.	Combine free tiers and paid providers behind one endpoint.
7	**Provider model catalogs**	Save multiple model IDs per provider while exposing one active model.	Replace a discontinued model without updating client code.
8	**Model health**	Track healthy, unavailable, unauthorized, rate-limited and untested states.	Distinguish a bad model slug from a broken provider account.
9	**Model aliases**	Stable virtual models with their own strategy, capability and provider rules.	A vision-router alias survives changes to the underlying model.
10	**Capability-aware routing**	Detect streaming, tools, vision, JSON mode, structured output and reasoning.	An image request avoids known text-only models.
11	**Flexible or strict capability mode**	Unknown support becomes a partial fallback or a hard exclusion.	Production schema workflows can require verified support.
12	**Six routing strategies**	Priority, Fastest, Round robin, Least used, Reliability and Smart.	Optimize each alias for order, speed, distribution or resilience.
13	**Provider-isolated failover**	Provider rejections and failures can move to the next candidate.	A missing Hugging Face model does not prevent Groq from answering.
14	**Configurable retries**	Retry statuses, network and malformed recovery, backoff and jitter.	Temporary 503 failures recover without a retry storm.
15	**Rate-limit cooldowns**	429 creates persistent cooldown state and honors Retry-After.	A provider reporting 60 seconds is skipped during that period.
16	**Circuit breakers**	Closed, open and half-open states with escalating recovery windows.	Repeated timeouts stop delaying every request.
17	**Recovery controls**	Clear cooldown, test recovery and reset circuit actions.	Verify a repaired provider without deleting its key.
18	**Provider quotas**	Daily and monthly request/token limits, warning thresholds and reset times.	Deprioritize a free tier at 80 percent and skip it at 100 percent.
19	**Reliability controls**	Provider timeout, streaming timeout, total deadline, max attempts and overrides.	Give a slow provider 10 seconds while another gets 30.
20	**Alias-level reliability**	Aliases can override timeout, deadline, attempts and order.	Chat uses a 15-second deadline while batch work allows two minutes.
21	**Request deduplication**	In-flight coalescing and short reuse of deterministic successes.	Two identical workers create one provider call and quota charge.
22	**Request correlation**	Safe client IDs or generated req_ IDs follow every attempt.	Trace one production failure across providers and logs.
23	**Rich routing headers**	Expose provider, model, alias, strategy, attempts, fallback, latency and dedup state.	Client logs reveal exactly which model served the request.

#	Feature	What exists today	Real-life value
24	**Full request timeline**	Chronological authentication, filtering, ranking, retry and delivery events.	Explain why a fourth-ranked provider was never attempted.
25	**Performance timing**	Router overhead, provider latency, retry delay, first token and throughput.	Separate gateway slowness from provider slowness.
26	**Analytics dashboards**	Provider, API/model, application, token, fallback and tool analytics.	Find the least reliable provider or highest-traffic client.
27	**Three Playground modes**	Routed request, exact cloud provider/model and exact Local LLM.	Test policy behavior separately from a specific model.
28	**Private Ollama nodes**	Pairing, discovery, signed inference and heartbeat monitoring.	Use a workstation for private prompts with optional cloud fallback.
29	**Local routing gate**	Keep a node connected while excluding it from normal routing.	Run diagnostics while Ollama is under maintenance.
30	**Local routing modes**	Normal, Prefer local and Local only.	Enforce private local processing or choose local-first savings.
31	**Local resource protection**	Concurrency, queue, input, token, first-token, idle and total limits.	Prevent a laptop from receiving twenty huge requests.
32	**Local lifecycle management**	Start, status, models, sync, logs, disconnect, rotate, revoke and delete.	Immediately revoke a lost computer or remove a retired node.
33	**Local-node security**	Single-use pairing, hashed credentials, Ed25519 signing and replay protection.	Knowing the ngrok URL is insufficient to call Ollama.
34	**Streaming**	Streaming across Chat Completions, Responses, Anthropic and Ollama paths.	Coding assistants display tokens as they arrive.
35	**Deployment flexibility**	Native Node server locally and Vercel-compatible hosted deployment.	Run personally or deploy as a shared product.

4. Complete routing pipeline

Routing is a sequence of gates. Providers that do not survive a gate never receive the request.

1. **Authenticate**
   Validate router key and load isolated account settings
2. **Deduplicate**
   Reuse a safe identical request or continue routing
3. **Resolve policy**
   Apply alias, capabilities, allowlist and reliability overrides
4. **Build pool**
   Combine cloud providers with eligible account-owned local models
5. **Filter**
   Remove capability, quota, cooldown, circuit and local-limit failures
6. **Create tiers**
   Full before partial; healthy quota before warning; local preference
7. **Rank**
   Priority, fastest, round robin, least used, reliability or smart
8. **Attempt**
   Apply per-provider timeout capped by the total deadline
9. **Recover**
   Immediate failover, backoff, cooldown or circuit transition
10. **Return and record**
    Convert response, add headers, usage, timeline and analytics

Core rule: a retry normally means trying the next provider, not repeating the same provider.

Candidate state vocabulary

Every provider evaluation is represented as candidate, incompatible, cooldown, quota-exhausted, limit-exceeded, circuit-open or half-open. Candidate ranks are assigned only after filtering and tier construction.

5. Every routing decision signal

These are the complete operational inputs currently capable of changing whether a model is selected, skipped or delayed.

Signal	Possible decision	Example
Router authentication	**Accept or reject before routing**	An invalid flm_ key returns 401 without contacting a provider.
Request validation	**Continue or return gateway error**	Malformed JSON stops because it is not a provider failure.
Deduplication	**Route, join in-flight or reuse completion**	Simultaneous deterministic requests share one upstream call.
Model alias	**Inherit or override policy**	vision-router requires vision and uses Reliability.
Alias allowlist	**Remove unapproved providers**	A legal workflow allows only local Ollama and one approved cloud.
Local-only mode	**Remove every cloud provider**	Confidential work fails rather than leaving the device.
Local routing gate	**Include or exclude a node**	Close production routing while preserving direct diagnostics.
Local availability	**Require online, endpoint, enabled model and capacity**	A saturated node is not selected.
Local limits	**Reject oversized local candidate**	A 4 MB prompt skips a node capped at 2 MB.
Capability support	**Full, partial or incompatible**	Verified vision ranks before unknown vision support.
Unknown capability mode	**Flexible fallback or strict exclusion**	Strict structured output excludes unverified models.
Quota health	**Healthy, warning or exhausted**	A provider at 85 percent moves behind one at 20 percent.
Cooldown	**Skip until expiry**	429 Retry-After 60 removes a provider for at least 60 seconds.
Circuit state	**Closed, open or half-open**	Three circuit failures open the provider, then one probe tests recovery.
Capability tier	**Full before partial**	Verified tool support is attempted before unknown support.
Local preference tier	**Prefer-local before comparable cloud**	The workstation serves first and Gemini remains fallback.
Routing strategy	**Order remaining candidates**	Reliability uses measured success history.
Provider timeout	**End one slow attempt**	Mistral is abandoned after its configured 30 seconds.
Maximum attempts	**Cap provider fan-out**	At three attempts, candidate four remains untried.
Total deadline	**Cap the entire operation**	Two 30-second timeouts can leave only 28 seconds for attempt three.
Client cancellation	**Abort routing**	Closing the caller connection prevents more attempts.
Streaming commitment	**Fail over only before output**	After tokens start, the stream ends rather than mixing models.

6. Routing strategies

Capability, health and quota gates happen before strategy ranking. A strategy therefore orders only the providers that remain eligible.

Strategy	How it decides	Best real-life use
Priority	**Saved provider order, then configured numeric priority.**	Deterministic primary provider with explicit backups.
Fastest	**Explore providers with little data, then choose lowest average latency.**	Interactive autocomplete and low-latency chat.
Round robin	**Rotate the start of the ordered list per router and alias.**	Spread development traffic across free tiers.
Least used	**Fewest attempts, then oldest last-use time, then priority.**	Keep trial accounts evenly exercised.
Reliability	**Exploration, then highest success score, latency and priority.**	Production automations where completion matters most.
Smart	**42% reliability, 20% latency, 18% quota, 12% lower usage and 8% priority, plus bonuses and penalties.**	General workloads using several limited providers.

What Smart does - and does not do

Smart combines 42% reliability, 20% latency, 18% remaining quota, 12% lower historical usage and 8% priority, then applies exploration bonuses and consecutive-failure penalties. It does not judge semantic answer quality or understand which provider gives the best answer to a particular prompt.

7. Provider outcomes and recovery

Provider failures are isolated wherever possible. Gateway-level authentication and request validation still stop before provider routing begins.

Provider outcome	Current router behavior
Valid 2xx	Return success and update timing, usage and health.
401	Authentication failure for that provider; immediate next candidate.
403	Access denied for that provider; immediate next candidate.
404 or missing model	Mark model or endpoint unavailable; immediate next candidate.
Capability rejection	Record runtime capability evidence; immediate next candidate.
Other non-transient HTTP rejection	Isolate to provider and fail over immediately without backoff.
Configured transient status	Backoff with optional jitter, then next provider.
429	Create cooldown, honor Retry-After and continue within recovery limits.
Any 5xx	Count toward circuit; configured retry codes also receive backoff.
Timeout or network failure	Count toward circuit and fail over when network recovery is enabled.
Malformed 2xx	Count toward circuit and fail over when malformed recovery is enabled.
Attempt budget exhausted	Stop with maximum_attempts_reached.
Total deadline exhausted	Stop with total_request_deadline_exceeded.
No remaining candidate	Stop with no_more_candidates.
All cooldown / quota / circuit restricted	Return a structured 429 or 503 with retry or reset timing.
No compatible provider	Return 400 no_compatible_provider before an upstream call.

Default reliability envelope

Setting	Default
Provider timeout	30 seconds
Streaming connection timeout	30 seconds
Half-open probe timeout	10 seconds
Total request deadline	90 seconds
Maximum provider attempts	3, configurable up to 20
Backoff	250 ms initial, 3 seconds maximum, multiplier 2, jitter enabled
Default retry HTTP codes	408, 409, 425, 429, 500, 502, 503 and 504

8. Worked routing examples

Example 1: Cloud failover

Groq returns 429 and enters cooldown. Mistral returns 503 and receives a circuit failure plus backoff. OpenRouter returns 200. The application succeeds and Analysis preserves all three attempts.

Example 2: Provider-specific model problem

Hugging Face returns 400 because the configured model does not exist. The router classifies the model as unavailable and immediately tries Groq. The bad model affects only Hugging Face.

Example 3: Capability-aware vision request

Gemini and Mistral are full matches, Kilo Code is partial and Groq is incompatible. Full matches are attempted first; Groq is never called for the image request.

Example 4: Free-tier protection

Groq is at 85 percent of its configured allowance while Mistral is at 20 percent. Both are compatible, but Mistral moves ahead of Groq to preserve remaining Groq capacity.

Example 5: Private local-first inference

Office Ollama is online in Prefer local mode. The local model serves first. If it fails before output begins, Gemini can still serve the request.

Example 6: Strict local-only privacy

Local only is enabled and the node is offline. Cloud providers may be configured, but they are excluded. The request fails clearly and the prompt never leaves the approved boundary.

Example 7: Attempt budget versus deadline

Maximum attempts is 20 but the total deadline is 90 seconds. Two 30-second timeouts leave roughly 28 seconds for the third provider. The deadline stops routing even though seventeen attempt slots remain.

9. Observability and operator workflow

Surface	What it answers	Example decision
Overview	**Is the router broadly healthy and receiving traffic?**	Notice success rate falling before users report it.
Providers	**Which cloud keys and local nodes are connected?**	Open the affected provider or jump to its Settings controls.
Playground - Router request	**Does the full production policy work?**	Confirm capability filtering and fallback selection.
Playground - Provider + model	**Can one exact cloud model respond?**	Diagnose an invalid model ID without other providers masking it.
Playground - Local LLM	**Can one exact Ollama model respond?**	Test a gated local node without adding it to production routing.
Analysis timeline	**Why did this exact request choose or skip providers?**	Find maximum_attempts_reached or total_request_deadline_exceeded.
Performance	**Where was time spent?**	Separate router overhead, provider latency and retry delays.
Analytics	**What patterns exist over many requests?**	Compare provider reliability, fallback paths, token use and client traffic.

Useful success headers

Responses can identify the request ID, selected provider and model, requested alias, routing strategy, capability match, attempt count, fallback usage, total latency, router latency, provider latency, retry delay and deduplication state.

10. Honest MVP boundaries

These limitations establish the current scope and create a credible future-work slide.

- Ollama is the only supported local runtime in v0.7.0.
- There is no /v1/embeddings endpoint yet.
- Responses and Anthropic requests are translated through chat-completions-compatible provider calls.
- Failover cannot occur after streamed output reaches the client.
- Deduplication is process-local rather than Redis-distributed.
- Unknown aliases currently use router-wide behavior instead of being rejected.
- Prompt size is not automatically converted into a cloud-provider context-window requirement.
- Cloud model maximum-output metadata is not currently enforced during ranking.
- Flexible capability mode may discover unsupported behavior only after an upstream attempt.

PRESENTATION ACCURACY NOTE

The current router implementation immediately fails over after generic upstream HTTP rejections. Two older worked scenarios in docs/ARCHITECTURE.md still describe the previous 422 and 501 stop behavior; use the current behavior documented in this PDF.

11. Suggested 12-slide presentation

This ordering moves from the problem to the architecture, proves the operational value and finishes with honest scope.

Slide	Title	Core message
1	**The problem**	Fragmented providers, separate credentials, quotas and outages.
2	**The product**	One self-hosted gateway, one router key and many execution targets.
3	**Architecture**	Control plane, routing data plane and private compute plane.
4	**Unified APIs**	Chat Completions, Responses/Codex and Anthropic Messages.
5	**Routing pipeline**	Alias, capability, health, quota, local policy and ranking.
6	**Six strategies**	Priority, speed, rotation, usage, reliability and smart scoring.
7	**Reliability**	Failover, backoff, cooldowns, circuits, attempts and deadlines.
8	**Efficiency**	Quota protection and safe request deduplication.
9	**Private Ollama**	Pairing, protected tunnel, routing modes and resource limits.
10	**Observability**	Playground, request timelines, headers and analytics.
11	**Live scenario**	Walk through one request moving across three providers.
12	**Scope and future**	Current limitations and the next logical extensions.

Closing statement

Free LLM Router turns many disconnected AI services and one private computer into a single observable, policy-controlled and failure-tolerant AI endpoint.

Repository references

- docs/MANUAL.md - product features, APIs and operational guide.
- docs/ARCHITECTURE.md - complete routing and scenario reference.
- docs/request-flow.md - detailed routing decision logic.
- docs/LOCAL_NODE_SECURITY.md - local trust boundary and signed requests.
- src/router.ts - current provider filtering, ranking, attempt and recovery behavior.